Chapter 2
Tokens and Regular Expressions

Lexical analysis, part 1: the first phase of a compiler and the declarative language we use to describe it

Compiler Construction Module

Based on Lecture 2, expanded

How to read this chapter

This chapter has one job: by the end of it you should be able to sit down with a language description, write a token specification for it, predict exactly which tokens a lexer will produce for any input, and explain *why*. Every rule here has a reason, and I will try to give you the reason before the rule. When you see a "Pause and check" box, actually stop and answer it before reading on; the answers are where the understanding happens.

1 Lexing: Breaking Characters into Tokens

1.1 Motivation: why the parser cannot work on characters

Let us start from what the *next* phase wants. A parser checks structure: "an `if`, then an open parenthesis, then an expression, then a close parenthesis, then a statement". It thinks in *words*. But a source file is not made of words. It is a flat sequence of characters, and most of them carry no structural meaning at all:

```
i f ␣ ( b ␣ = = ␣ 0 ) ␣ a ␣ = ␣ " h i "
```

If the parser had to work at this level, every grammar rule would have to say "`i` followed by `f`, followed by any number of spaces, tabs or comments, followed by ...". The grammar for a real language would be enormous, and every rule would repeat the same whitespace-handling clutter. So we do what engineers always do with a messy problem: we split it. One component, the **lexer**, deals with characters and produces words. The parser then only ever sees words.

An analogy: when you read this sentence, you do not consciously assemble letters into words. Your eyes deliver whole words to the part of your brain

History. The word "lexeme" comes from linguistics: the smallest unit of meaning in a language. `lex` (1975, Mike Lesk and Eric Schmidt at Bell Labs) was the first widely used lexer generator; `flex` and JFlex are its descendants.

that handles grammar. Lexing is that first, almost unconscious step, made explicit.

The split has a second benefit that is easy to underestimate. Every character-level nuisance of a language (escape sequences in strings, the seven ways to write a floating-point literal, nested comments, tabs versus spaces) lives in one small component. Get that component right once and the rest of the compiler never thinks about characters again.

Concept: Tokens and token classes

The lexer breaks the input character stream into **tokens**. Each token belongs to a **class**. The usual classes are:

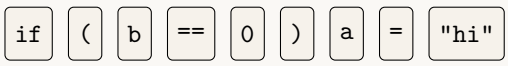
- **identifiers**: names the programmer chose, such as `x` or `item_count`
- **keywords**: names the language reserved, such as `if` or `return`
- **literal constants**: values written directly, such as `23`, `-1.23e-5`, `"Hello!"`
- **operators**: `+`, `%`, `==`
- **punctuation**: `(`, `)`, `;`, `,`

Whitespace and comments are usually not tokens at all. They separate tokens and are then discarded, because no later phase needs them.

Why do we classify at all, rather than just splitting into words? Because the parser's rules are written in terms of classes, not specific words. The rule for an assignment says "an *identifier*, then `=`, then an expression"; it does not list every identifier the programmer might invent. Classes are what make a finite grammar describe an infinite set of programs.

Worked example: A natural tokenisation

Input: `if (b == 0) a = "hi"`



Nine tokens. Look closely at three decisions the lexer made silently:

- The four spaces produced nothing. They did their job (separating `if` from `(`, for instance) and vanished.
- `==` became *one* token, not two `=` tokens. The lexer had to look at the second character before deciding.
- The quotes around `hi` are part of the string token's *text*, but the token's *value* is just `hi`. Delimiters tell the lexer where the string is; they are not part of the string.

1.2 A caveat: comments are not always noise

I said comments are discarded. That is true for a lexer feeding a compiler, because the compiler genuinely does not care what a comment says. But a lexer is a general tool, and some tools are built *specifically* to read comments. Javadoc, Doxygen and Python's docstring tools generate documentation from

specially formatted comments:

```
/**
 * Computes the square root of a number.
 * @param x the input value
 * @return the square root of x
 */
double sqrt(double x) { ... }
```

To Javadoc, the `/** ... */` block is structured data: a description, a `@param` tag with a name and text, a `@return` tag. A lexer built for that job has to tokenise the comment's contents rather than skip them. The general lesson: *what counts as a token depends on who consumes the tokens.* There is no universal "correct" tokenisation of a file; there is only the tokenisation that the next phase needs.

1.3 What a token actually is: a small data record

So far I have drawn tokens as boxes with text in them. Let us make that precise, because the rest of the chapter depends on it. **A token is a small data record**, the kind of thing you would write as a struct in C, a class in Java, or a tuple in Python. It is produced by the lexer and consumed by the parser. Every token has three fields:

Field	What it holds	Example for the source text `count`
class (or kind, or type)	which category of token this is: identifier, keyword, integer literal, ...	`IDENTIFIER`
attribute (or value)	the specific information this particular token carries, if any	the string `"count"`
position	where in the source file the token started	line 3, column 7

Why three fields and not just the text? Think about what the parser and later phases need.

- The **class** is what the parser looks at. Its grammar rules say things like "an identifier, then `=`, then an expression". The parser asks "is the next token an identifier?", not "is the next token the word `count`?".
- The **attribute** is what the later phases look at. The type checker needs to know *which* variable; the code generator needs to know *which* number to load. For an integer literal the attribute is the integer itself; for an identifier it is the name; for a string literal it is the string's contents. Some tokens need no attribute at all: an `IF` keyword or a `(` carries no information beyond its class.
- The **position** is for error messages, as we will see in a moment.

Interface / API: The same token record in three languages

The lexer is part of the compiler, so it is written in whatever language the compiler is written in (Chapter 1 discussed that choice). The course's examples use Java, and the lexer generator we will meet in Section 4 is JFlex, which produces Java. But nothing here is Java-specific. Here is one and the same token, the identifier `count` at line 3 column 7, as a data record in three languages.

C

```c
struct Token {
  enum Kind kind;
  char *text;
  int line, col;
};
struct Token t =
  { IDENTIFIER,
    "count", 3, 7 };
```

Java

```java
class Token {
  TokenType kind;
  Object value;
  int line, col;
}
new Token(
  TokenType.ID,
  "count", 3, 7);
```

Python

```python
# a plain tuple:
# (kind, value,
#  (line, col))


("ID", "count",
 (3, 7))
```

Notation. Writing out the full record every time is tedious, so from here on I use a compact notation: the class in capitals, followed by the attribute in parentheses if there is one. So `ID("count")` means "a token of class identifier whose attribute is the string `count`"; `INT(10)` means "an integer-literal token whose attribute is the number ten"; and plain `IF` means "a keyword token of class `IF`, no attribute". Positions are left out of the notation unless they matter. When you see `ID("sum") NUMBER(1)` later, read it as a sequence of two token records.

Key insight. Throughout this chapter, `ID("count")`, `INT(10)` and `IF` are shorthand for token records: class in capitals, attribute in parentheses, no parentheses when there is no attribute.

The slide's version of the same idea

The lecture slide shows tokens being built like this, in Java:

```java
new Identifier("a")         // identifier a
new Number(10.0)            // floating literal 10
new StringLit("hello\n")    // string literal "hello\n"
```

Read `new Identifier("a")` as "construct a token record of class Identifier whose attribute is `a`". This design uses one Java class *per token class* (`Identifier`, `Number`, `StringLit`, presumably `Keyword`, and so on) instead of one `Token` class with a `kind` field. Both designs are common; they store the same information. The slide's design lets each class give its attribute the right type (a `double` for `Number`, a `String` for the others).

Now look at the string literal, because it illustrates a principle. In the source file the programmer typed *nine* characters: a quote, the five letters `hello`, a backslash, the letter `n`, and a closing quote. The token's attribute is a *six*-character string: `hello` followed by an actual newline character. The lexer has already decoded the escape sequence and stripped the delimiters. Why do this here? Because if the lexer did not, every later phase that touches strings

Key insight. The attribute is the *interpreted* value, not the source text. The lexer is where escape sequences and number formats are decoded, once, so no later phase has to.

(the type checker, the constant folder, the code generator) would each need its own escape-sequence decoder, and they would inevitably disagree. Decode once, at the boundary.

Position

Each token also records its **position**: file, line, and column (or a column range). Why? Because errors are usually detected long after lexing, by phases that no longer have the source text. When the type checker complains, it can only say *where* because the token remembered:

```
main.c:42:8: error: undeclared variable 'x'
```

The type checker never saw the file. It saw a token whose position field said line 42, column 8, and it printed that.

Worked example: Pause and check: position tracking

Given these three lines (1-based line and column numbering):

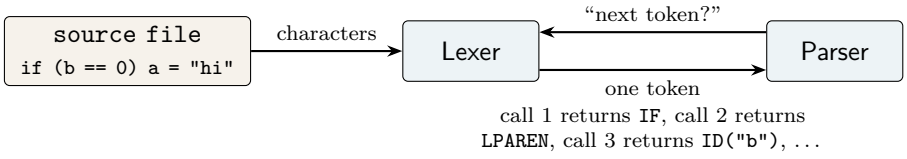
```
x = 1
y = 2
z = x +
```

What position does the lexer report for the token `y`, and for the final `+`?

`y` is the first character of line 2, so line 2, column 1. For the `+`, count every character on line 3, spaces included: `z` is column 1, the space is 2, `=` is 3, space 4, `x` 5, space 6, `+` 7. So line 3, column 7.
The lesson is in the method: a real lexer keeps a line counter and a column counter, advances the column on *every* character it reads (including spaces and characters inside comments), and on a newline increments the line and resets the column to 1. It records the counters at the moment a token *starts*. Trying to compute positions afterwards, by searching the source for the token's text, fails as soon as the same text appears twice.

1.4 How the lexer is used: who calls it, and when

Before looking at the interface, let us be clear about where the lexer sits and who talks to it. The lexer is a component *inside the compiler*. It runs when you compile, not when your program runs. It is given the source file and it hands tokens to the parser. The parser is in charge: it asks the lexer for one token, does something with it, asks for the next, and so on until the file is finished.



So the lexer is not a program you run by hand on a file. It is a function (or object) that the parser calls repeatedly. That is what the interface below describes.

1.5 The interface: what a lexer looks like from outside

The slide gives the lexer's interface as a Java type signature. If you do not know Java, here is how to read it, one piece at a time.

Interface / API: The shape of a lexer

```
Iterator<Token> lex(Reader input);
```

This declares a function called `lex`. Reading right to left:

- `Reader input`: the one parameter. A `Reader` is Java's name for "a source of characters that you read one at a time", such as an open file. So: the lexer takes the source text as input.
- `Iterator<Token>`: the return type. An `Iterator` is Java's name for "something you can ask for the next item, repeatedly". It has two operations: `hasNext()` (is there another?) and `next()` (give me the next one). `<Token>` says the items are `Token` records. So: the lexer returns a stream of tokens that the caller pulls one at a time.

In plain words: *give the lexer a character stream, and it gives you back a token stream.*

Here is what using it looks like, in Java and in Python, so the idea is not tied to one language. The parser does exactly this, except that instead of printing each token it decides what to do with it.

Java

```
Reader in = new FileReader(f);
Iterator<Token> toks = lex(in);
while (toks.hasNext()) {
  Token t = toks.next();
  System.out.println(t);
}
// IF  LPAREN  ID("b")  EQ ...
```

Python, same idea

```
text = open(f).read()
for t in Lexer(text).tokens():
    print(t)
# ('KEYWORD', 'if', (1,1))
# ('PUNCT', '(', (1,4))
# ('ID', 'b', (1,5)) ...
```

(The Python `Lexer` class is the one written out in full in Section 2.5; `tokens()` is a generator, which is Python's version of an iterator.)

Two design choices are hiding in that one-line signature, and both are deliberate.

Streams, not arrays. The lexer does not read the whole file and return a list of all tokens. It produces one token each time `next()` is called, then stops and waits. This keeps memory use constant regardless of file size, and it means an error on line 3 is reported without first lexing line 4000. It also lets interactive tools (a REPL, an IDE) start parsing a line before you finish typing the next.

In the real world. Pull-based (`Iterator`) lexers are also why a syntax error early in a file is reported almost instantly even for a huge file: nothing after the error was ever lexed.

Errors are part of the interface. What should `next()` do when the input contains a character that begins no legal token, say a stray `#` or `@` in a language that has neither? It cannot pretend. The options are an `ErrorToken` variant in the stream (so the parser can decide how to recover), a thrown exception, or an error callback. Whichever you pick, it must be designed in from the start. A lexer that silently skips unknown characters produces baffling parse errors three tokens later.

Summary

Lexing converts characters into a stream of classified tokens. Each token carries its class, an *interpreted* attribute (the decoded string, the numeric value), and the position where it started. Whitespace and usually comments are discarded. The interface is a pull-based iterator over tokens, with a defined way to report input that matches no token.

2 Hand-Written Lexers and Why They Go Wrong

2.1 Motivation: it looks easy, and that is the problem

Nothing about lexing looks hard. Read characters, group them, done. So impatient programmers write it by hand, and the result is a major source of real security vulnerabilities, especially in unsafe languages. Think about why: a lexer is the component that runs on raw, untrusted input, before any validation has happened. An off-by-one error in ad-hoc scanning code is exactly how a program reads past the end of a buffer.

In the real world. Many CVEs in image parsers, protocol decoders and configuration readers are lexing bugs: a hand-written scanner that trusted a length field or ran one character past the end of a buffer.

The core technical difficulty is this: **tokens overlap**. When the lexer sees the letter `i`, it does not yet know whether it is looking at the keyword `if`, the keyword `in`, or the identifier `index`. It must read more before deciding. And, crucially, it often needs to look at the character *after* the token to know the token has ended. The identifier `sum` ends because the next character is `+`, and `+` is not part of any identifier. But to know that, the lexer had to look at the `+`, which belongs to the *next* token.

2.2 Bug 1: `read()` discards the lookahead

Here is the natural first attempt at reading an identifier, and the correct version beside it.

Broken

```
while (true) {
  int c = input.read();
  if (!isIdChar(c)) break;
  id.append((char) c);
}
```

Fixed

```
while (isIdChar(input.peek())) {
  id.append(input.read());
}
```

The difference is one method call. `read()` returns the next character *and removes it from the stream*. `peek()` returns the next character *and leaves it there*. The broken version reads a character, then asks whether it wanted it. By then it is too late: the character is gone.

Let us trace the broken version on the input `sum+1`, character by character:

Step	`read()` returns	`isIdChar?`	Action
1	`s`	true	`id.append('s')`
2	`u`	true	`id.append('u')`
3	`m`	true	`id.append('m')`
4	`+`	false	`break`; but `+` is already consumed and lost

The identifier itself comes out fine: `sum`. But the `+` was removed from the stream, inspected, found to be unwanted, and then simply dropped on the floor. When the lexer is next asked for a token, the stream now starts at `1`. The final output is `ID("sum") NUMBER(1)`. The operator has vanished, and the parser will later report something like "unexpected number after identifier", pointing at a line that looks perfectly fine.

Warning / Gotcha: Why this bug hides so well

Run the same code on `sum + 1` with spaces. Now the character consumed and discarded at step 4 is a *space*, which the lexer was going to throw away anyway. The output is correct. The bug is invisible. It only shows up when a real token immediately follows another with no separator between them, and that is exactly the case your quick tests will not include. This is the pattern of most lexer bugs: correct on nicely formatted input, wrong on dense input.
The fix is a slightly richer input abstraction: `peek()` lets you inspect the next character without committing to it. Only call `read()` once you know you want the character.

2.3 Bug 2: committing too early on `=`

The same mistake in a different costume. A hand-written lexer reads one character and, the moment it sees `=`, emits an `ASSIGN` token, without looking at what comes next. On `x==y` it produces

```
ID("x") ASSIGN ASSIGN ID("y")      (wrong: should be ID EQ ID)
```

The parser will then complain that you cannot assign to an assignment. Same principle, same fix: after seeing `=`, `peek()` at the next character. If it is also `=`, consume both and emit `EQ`; otherwise emit `ASSIGN` alone.

Notice that the two bugs are mirror images. Bug 1 consumed a character it should have left. Bug 2 *stopped* before a character it should have taken. Both come from deciding what the token is before having seen enough. The general principle: **a lexer must always know how far ahead it needs**

to look before it commits. We will see that a lexer generator handles this automatically: give it `"="` and `"=="` as two rules and it always prefers the longer match, with no hand-written lookahead code at all.

2.4 Lookahead versus backtracking

`peek()` is **lookahead**: seeing the next character without consuming it. One character of lookahead solves both bugs above. But is one character always enough? No.

Worked example: Where one character of lookahead fails

Suppose the specification has two token rules: `abc`, and `(abc)*d` (any number of repetitions of `abc`, then a `d`). The input is `abcabcx`.
The lexer reads `a`, `b`, `c`. At this point rule 1 matches. Should it stop and emit `abc`? It cannot decide yet, because rule 2 might match a *longer* token: if the input continues `abcd` or `abcabcd`, rule 2 wins. So it must keep reading. It reads `a`, `b`, `c` again: still consistent with rule 2. It reads `x`. Only now is rule 2 definitely dead.
Six characters have been consumed chasing a token that does not exist. The correct answer was the token `abc`, of length 3, which the lexer passed three characters ago. Recovering it means *rewinding* the input all the way back to position 3, then continuing from there with `abcx`.

That rewinding is **backtracking**: after consuming several characters, putting them back and trying again. It is strictly stronger than lookahead, because the amount to rewind is not bounded (imagine the input `abcabcabcabcabcx`). A lexer's input abstraction therefore needs two operations beyond `read()`: `peek()` for one-character lookahead and some form of `mark()`/`reset()` or `unread()` for arbitrary backtracking. The course's `EasyIO.Scanner` class is one such abstraction. The next chapter's scan-and-backtrack algorithm is built on exactly this: remember the position of the last complete match, keep scanning while any rule could still match, and rewind to the remembered position when none can.

2.5 Implementation deep-dive: a minimal correct hand lexer

We are about to replace hand-written lexers with generated ones. Even so, reading one correct hand-written lexer is worth it, because generated code does the same things, only mechanically. This one handles identifiers, keywords, integers, `=` versus `==`, single-character punctuation, whitespace, and tracks line and column. Read it with the two bugs in mind and notice where each is avoided.

```
1  class Lexer:
2      def __init__(self, text):
3          self.s, self.i = text, 0
4          self.line, self.col = 1, 1
5
6      def peek(self, k=0):                    # lookahead, never consumes
```

```
        j = self.i + k
        return self.s[j] if j < len(self.s) else ''

    def read(self):                          # consume one char, track position
        c = self.s[self.i]; self.i += 1
        if c == '\n': self.line += 1; self.col = 1
        else:         self.col += 1
        return c

    def tokens(self):
        while self.peek():
            c, pos = self.peek(), (self.line, self.col)   # position BEFORE consuming
            if c.isspace():
                self.read()                  # discard
            elif c.isalpha() or c == '_':
                w = ''
                while self.peek().isalnum() or self.peek() == '_':
                    w += self.read()      # peek BEFORE read (avoids bug 1)
                kind = 'KEYWORD' if w in {'if', 'return'} else 'ID'
                yield (kind, w, pos)
            elif c.isdigit():
                n = ''
                while self.peek().isdigit(): n += self.read()
                yield ('INT', int(n), pos)    # attribute is the decoded value
            elif c == '=':
                self.read()
                if self.peek() == '=':        # check before you commit (avoids bug 2)
                    self.read(); yield ('EQ', '==', pos)
                else:
                    yield ('ASSIGN', '=', pos)
            elif c in '+-*/();':
                yield ('PUNCT', self.read(), pos)
            else:
                raise SyntaxError(f"{self.line}:{self.col}: bad char {c!r}")
```

Walk through what happens on `if x==10` so the structure is clear. The outer loop peeks `i`, sees a letter, and enters the identifier branch. The inner loop reads `i`, `f`, then peeks the space, which is not alphanumeric, so it stops *without consuming the space*. The word `if` is in the keyword set, so the token is `(KEYWORD, 'if', (1,1))`. Back in the outer loop, the space is discarded. Then `x` gives `(ID, 'x', (1,4))`. Then `=`: we read it, peek, see another `=`, read that too, and emit `(EQ, '==', (1,5))`. Then the digits: `(INT, 10, (1,7))`, with the attribute already converted to the integer ten. Notice that keywords are recognised by first lexing an identifier and then checking a set. That is the standard trick in hand-written lexers; a generator does it differently, as we will see.

Worked example: Pause and check: a third variant of the same bug

A hand-written routine for string literals does this: after the opening quote, repeatedly read a character and append it to a buffer, and stop when the character just appended is a quote. What is wrong with it?

It appends the closing quote *before* checking whether it was the closing quote, so the delimiter ends up inside the token's value. On `"hi"` the buffer goes `h`, `hi`, `hi"`, and the token contains three characters. The fix is the same shape as always: peek at the next character; if it is the closing quote, consume it and stop *without* appending. And add an end-of-input

check, or an unterminated string loops forever.

Summary

Hand-written lexers fail in two recurring ways: consuming a character before knowing it belongs to the current token, and committing to a token before checking whether a longer one is possible. Both are fixed by *peek before read*. Some specifications need more than one character of lookahead, which requires backtracking to the last complete match. Better still: do not hand-write the lexer at all.

3 Regular Expressions, Precisely

3.1 Motivation: describe the tokens, do not code them

Instead of writing lexer code, we will *describe* the legal tokens in a small notation and let a tool synthesise the lexer from the description. This is the course's theme in miniature: a precise, declarative specification, compiled down to efficient imperative code.

Key insight. A regex is a *specification*, not a program. The lexer generator is the compiler that turns the specification into a program.

Two reasons this is better than hand-writing. First, it is easier to get right: the specification is a few lines you can read in full, and the generated code handles lookahead and backtracking uniformly. Second, it is necessary, because real tokens are intricate. Here are some legal C and Java floating-point literals:

```
2.0   .1e0   20.e99   1e5   0x1.8p3
```

A hand-written `if`-cascade that accepts exactly these and nothing else runs to forty lines and will have bugs. The regular expression is two lines.

3.2 What a regular expression means

The most important idea in this section is easy to state and easy to forget: **a regular expression is a name for a set of strings**. It does not "do" anything. When I write $a\,b$, I am not giving an instruction; I am naming the set that contains the single string "ab". When I write $(a \mid b)^*$, I am naming an infinite set. The set a regular expression R names is called its **language**, written $L(R)$. To ask "does the string s match R?" is to ask "is $s \in L(R)$?".

Concept: The core syntax and its meaning

Assume a and b are ordinary characters of the alphabet. The entire core notation is five constructs:

Syntax	Language	In words
a	$\{$ "a" $\}$	the one-character string a, and nothing else
ε	$\{$ "" $\}$	just the empty string (a set with one element, not the empty set)
$R \mid S$	$L(R) \cup L(S)$	anything that matches R, or anything that matches S
$R\,S$	$\{\, xy \mid x \in L(R),\ y \in L(S) \,\}$	something from R immediately followed by something from S
R^*	$L(\varepsilon) \cup L(R) \cup L(RR) \cup \cdots$	zero or more things from R, one after another

Precedence, highest first: star, then concatenation, then union. So $ab^* \mid c$ means $(a(b^*)) \mid c$. Use parentheses whenever in doubt.

Let me unpack the three that cause trouble.

Union $R \mid S$ is a choice. $a \mid b$ is the set {"a", "b"}: two strings, each of length one. It is *not* the string "a|b" and it is not "a or b" in some vague sense; it is a set with exactly two members.

Concatenation RS glues. Take every string in $L(R)$, take every string in $L(S)$, and form all the ways of writing one followed by the other. If $L(R) = \{$"a", "b"$\}$ and $L(S) = \{$"x"$\}$, then $L(RS) = \{$"ax", "bx"$\}$. Concatenation distributes over union in exactly the way multiplication distributes over addition, which is why the notation looks algebraic.

Star R^* is the only construct that produces infinite sets, and it is the one students most often misread. R^* means: zero copies of something from R, or one copy, or two, or any number, written one after another. The "zero copies" case is why the empty string is always in $L(R^*)$, for any R. If you want "one or more", star alone is not it.

Worked example: Reading regular expressions, slowly

Regular expression	Language, and how to see it
$a\varepsilon$	{"a"}. Concatenating with the empty string changes nothing.
$a \mid b$	{"a", "b"}.
$(ab)^*$	{"", "ab", "abab", "ababab", ...}. Note "a" alone is *not* in it; the unit being repeated is "ab".
ab^*	{"a", "ab", "abb", ...}. Star binds tighter than concatenation, so only b repeats. Compare with the previous row.
$(a \mid \varepsilon)\, b$	{"ab", "b"}. "An a, optionally, then a b."
$(a \mid b)^*\, abb$	every string over $\{a, b\}$ that ends in "abb". The prefix can be anything, including empty.

For the last one: "abb", "babb", "aabb" are members. "ab" is not (it does not end in abb). "abbb" is not either: it ends in "bbb". A common mistake is to think "abbb" qualifies because it *contains* abb. The expression requires abb at the *end*, with nothing after it.

Worked example: Pause and check

Which of these strings are in $L((a \mid b)^*a)$: "a", "b", "ba", "" (empty), "aab"?

The expression says "anything over $\{a, b\}$, then an a". So: any string that ends in a. "a" yes (empty prefix, then a). "b" no. "ba" yes. "" no (there must be a final a). "aab" no (ends in b).

3.3 Syntactic sugar

Writing "optional" as $(R \mid \varepsilon)$ and "one or more" as RR^* gets tedious. Every lexer generator therefore provides shorthand. The important point is that none of it adds power: each abbreviation is *defined* in terms of the core five, so anything you can say with the sugar you could already say without it.

Mechanism: Shorthand, all desugared to the core five

Syntax	Desugars to	Meaning
R?	$R \mid \varepsilon$	optional: zero or one R
R+	$R\,R^*$	one or more R
`[abc]`	$a \mid b \mid c$	character class: any one of the listed characters
`[a-z]`	$a \mid b \mid \cdots \mid z$	a range: any one character between the bounds
`[^ab]`	(every alphabet character except a, b, unioned)	negated class: any one character *not* listed
`.`	roughly `[^\n]`	any one character except newline
`"if"`	$i\,f$	a literal string: the characters in sequence

Three things to keep straight, because they cause most errors in practice:

- A character class matches *exactly one* character. `[0-9]` matches "7" but not "77". For a run of digits you need `[0-9]+`.
- `[^ab]` matches one character that is not a or b. It *does* match a newline, a space, or a quote. `[^]` with nothing after the caret matches any single character whatsoever, newline included; that is the standard way to say "anything" inside a comment rule.
- `.` excludes the newline. A rule like `"//".*` for a line comment works precisely because `.` stops at the end of the line.

3.4 Abbreviations: naming pieces of a regex

Real token specifications get long. Lexer generators let you name a piece of a regex and reuse it, including inside other named pieces:

```
digit  = [0-9]
posint = [1-9]{digit}*
```

`posint` expands, by textual substitution, to `[1-9][0-9]*`: a non-zero digit followed by any digits. It describes positive integers without leading zeros. The named version is simply easier to read and, more importantly, easier to change: if the language later allows underscores in numbers, you edit `digit` once.

Think of an abbreviation as a macro. When the generator sees `{digit}`, it pastes in the definition. That mental model tells you immediately what is and is not allowed.

Warning / Gotcha: Recursive abbreviations are illegal

```
x = {x}a |      // illegal: never finishes expanding
```

Try to expand it by pasting: `{x}a` becomes `({x}a | )a`, which becomes `(({x}a | )a | )a`, and so on forever. The substitution never bottoms out into a finite regular expression. In this case it also adds nothing, because `x = a*` says exactly the same thing.
But the ban is not just about this example. Recursive definitions are precisely what a context-free grammar has and a regular expression does not. If `x` could refer to itself, you could write `x = "(" {x} ")" |` , which describes balanced parentheses, which is *not* a regular language. Allowing recursion would silently lift the lexer from Type 3 (regular, recognised by a finite automaton) to Type 2 (context-free, needing a stack). Tokens never need that power, and paying for it would make lexing slower and the generator far more complex. So the rule is: abbreviations may refer to earlier abbreviations, never to themselves, directly or indirectly.

3.5 How to build a regex for a token: a method

Students often stare at a token description and do not know where to start. Here is a method that works every time.

Technique: Building a regex, step by step

1. Write down three or four *examples* of the token and two or three near-misses that must be rejected.
2. Split the token into *parts* that appear left to right. Name each part.
3. For each part, decide: exactly one character (`[...]`), one or more (`+`), zero or more (`*`), or optional (`?`).
4. Concatenate the parts in order. Where there is a choice between alternatives, use `|` with parentheses.
5. Check every example and every near-miss against your regex by hand.

Worked example: Applying the method: scientific notation

Write `sci` for numbers like `2.5e10`, `3E-4`, `42.0e0`, given `digit = [0-9]` and `decimal = {digit}+ "." {digit}+`.
Step 1, examples. Accept `2.5e10`, `3.0E-4`, `42.0e+0`. Reject `2.5e` (no exponent digits), `e10` (no mantissa), `2.5f3` (wrong letter).
Step 2, parts. A decimal, then the letter `e` or `E`, then an optional sign, then one or more digits.
Step 3, quantities. Decimal: exactly one (it is already a complete piece). Letter: exactly one of two choices. Sign: optional. Digits: one or more.
Step 4, assemble.

```
digit   = [0-9]
decimal = {digit}+ "." {digit}+
sci     = {decimal} ("e"|"E") ("+"|"-")? {digit}+
```

Step 5, check. `3.0E-4`: `3.0` is a decimal, `E` is allowed, `-` is the optional sign, `4` is one digit. Accept. `2.5e`: after `e` we need at least one digit and there is none. Reject. Good.
The optional sign `("+"|"-")?` is the $R?$ sugar, which desugars to `("+"|"-"|)`. Note that `3E-4` from the lecture slide would actually be rejected by this `sci`, because `decimal` requires a decimal point. Whether that is a bug depends on the language you are specifying, which is exactly why step 1 matters: write the examples down first.

Worked example: Applying the method: string literals with escaped quotes

A string starts and ends with `"`. Between the quotes it may contain any character except an unescaped `"`, or an escape sequence `\X` for any character X. Example: `"he said \"hi\""`.
Parts. Opening quote; then a repeated body; then closing quote. The body is the interesting part: each unit of it is *either* an ordinary character *or* an escape.

```
string = "\""  ( [^"\\]  |  "\\" . )*  "\""
```

- `[^"\\]`: one ordinary character, which is anything that is not a quote and not a backslash. (Inside the class, `\\` is how you write a single backslash.)
- `"\\" .`: a backslash followed by *any* one character. This single alternative covers `\"`, `\\`, `\n`, `\t`, and every other escape.
- The star lets the body be any mix of the two kinds, including empty (so `""` is a valid string).

Why exclude backslash from the ordinary-character class? Because if a backslash could match as ordinary, then in `"a\"b"` the lexer could treat `\` as ordinary and then see `"` as the closing quote, ending the string early. Forcing every backslash through the escape alternative guarantees the character after it is swallowed with it.
Trace `"he said \"hi\""`. Opening quote. `h`, `e`, space, `s`, `a`, `i`, `d`, space: all ordinary. Then `\`: not ordinary, so it must be the escape alternative, which consumes `\` *and* the following `"` together. `h`, `i`: ordinary. `\"`: escape again. Then `"`: not ordinary, not a backslash, so the body ends and this is the closing quote. One token.
Meta-wrinkle: writing this regex requires escaping `"` and `\` twice over, once for the language being lexed and once for the spec syntax itself. Expect to get it wrong the first time; test it.

Summary

A regular expression names a set of strings. Five constructs (character, ε, union, concatenation, star) are the whole language; `?`, `+`, character classes and `.` are sugar defined in terms of them. Star always includes the empty string. A character class matches one character. Named abbreviations are

macros and must not be recursive, because recursion is exactly the step from regular to context-free. Build a regex by listing examples, splitting into parts, choosing a quantity for each part, and checking.

4 Lexer Generators in Practice

4.1 Conceptual overview

A lexer generator (JFlex for Java, `lex`/`flex` for C, `ocamllex`, `re2c`) reads a **specification file** and writes a source file containing a complete lexer. The specification is a list of **rules**, each pairing a regular expression with an **action**: a snippet of ordinary code that runs when the regex matches. The action usually constructs and returns a token.



The generated lexer's `next()` method does the following, every time it is called: starting at the current input position, find the rule whose regex matches the longest prefix of the remaining input, consume that prefix, and run the rule's action. If the action returns a token, `next()` returns it; if the action returns nothing (as for whitespace), `next()` loops and tries again from the new position.

4.2 Technical specification: a JFlex spec

```
1  Whitespace = [ \t\f\r\n]
2  Letter     = [a-zA-Z]
3  Digit      = [0-9]
4  Identifier = {Letter}({Digit}|{Letter}|_)*
5  Integer    = "0" | "-"?[1-9]{Digit}*
6  Float      = {Digit}+ "." {Digit}+
7
8  %%
9
10 {Whitespace}  { /* ignore */ }
11 "if"          { return new Token(TokenType.IF); }
12 {Identifier}  { return new Token(TokenType.ID, yytext()); }
13 {Integer}     { return new Token(TokenType.INT, Integer.parseInt(
       yytext())); }
14 {Float}       { return new Token(TokenType.FLOAT, Float.parseFloat(
       yytext())); }
```

Let us read it line by line, because every line teaches something.

- Lines 1–6, above the `%%`, are abbreviations. `Identifier` says: a letter, then any mix of digits, letters and underscores. So `x1` and `item_count` are

identifiers; `1x` is not (it starts with a digit); `_x` is not either, under *this* spec, because the first character must be a letter.

- `Integer` is either the single character `0`, or an optional minus sign, a non-zero digit, and any further digits. So `0`, `42`, `-7` are integers; `007` is not one token (we will see what happens to it later).
- `Float` is digits, a dot, digits. Both sides are `+`, so `1.` and `.5` are not floats here. And there is *no* optional sign, unlike `Integer`. Keep that in mind.
- Line 10: whitespace matches and the action does nothing, so no token is produced and the lexer moves on.
- Line 11: the literal string `"if"`. Its action returns a token with no attribute.
- Lines 12–14: `yytext()` returns the exact text just matched, as a string. The identifier action stores that text as the attribute. The integer action *parses* it, so the attribute is the number, not the digits. This is the decode-once principle from Section 1 in action.

4.3 Implementation deep-dive: how conflicts are resolved

Now the central question. On input `if`, both the rule `"if"` and the rule `{Identifier}` match. On input `iffy`, both match a prefix (the first two characters), and `{Identifier}` also matches all four. Which rule fires, and how much does it consume? Every lexer generator answers with the same two rules, applied strictly in this order.

Mechanism: The two tie-breaking rules

1. **Longest match (maximal munch).** Consider every rule that matches *some* prefix of the remaining input. Among all those matches, take the one that consumes the *most* characters. This decides how long the token is.
2. **Rule order.** Only if two or more rules match the *same, longest* length does order matter: the rule written *first* in the specification wins. This decides which token class it is.

Why longest match, and not, say, first match? Because it is what programmers mean. When you write `count`, you mean one identifier, not the keyword `co` followed by `unt`. When you write `<=`, you mean less-or-equal, not less-than followed by assignment. Longest match encodes the convention that a token extends as far as it possibly can.

Why does order only break ties? Because if order came first, you would have to write rules in a very careful sequence, and a keyword rule written above the identifier rule would chop `iffy` into `if` and `fy`. With longest match first, order only matters when the input is *exactly* a keyword, which is exactly when you want the keyword rule to win.

Worked example: Keyword or identifier?

Suppose `"in"` is a keyword and identifiers match `[a-zA-Z_][a-zA-Z0-9_]*`. On input `infinity`, does the lexer pro-

duce `IN ID("finity")` or `ID("infinity")`?

One `ID("infinity")`. Apply rule 1: `"in"` matches a prefix of length 2; the identifier rule matches a prefix of length 8. Eight beats two, so the identifier rule wins and consumes all eight characters. Rule 2 never comes into play because the lengths differ.
When *would* rule 2 matter? On the input `in` by itself (followed by a space, say). Now both rules match exactly 2 characters. It is a tie, so the rule written first wins. If the keyword rule is above the identifier rule, you get `IN`; if below, you get `ID("in")` and your keyword is unreachable. This is why every specification lists keywords *before* the identifier rule: order settles the exact-length tie in the keyword's favour, while longest match protects `intake`, `inner` and `infinity` from being carved up around a keyword that merely starts the same way.

Worked example: A full trace: longest match at every step

Let us run the JFlex spec above on the input line `if x1>=10.5` (with `">="` and `">"` added as two more rules) and watch both tie-breaking rules operate.

Remaining input	Candidate matches (rule: length)	Result
`if x1>=10.5`	`"if"`: 2, `Identifier`: 2	tie at 2, `"if"` is first: `IF`
` x1>=10.5`	`Whitespace`: 1	discarded
`x1>=10.5`	`Identifier`: 2 (`x1`; stops at `>`)	`ID("x1")`
`>=10.5`	`">"`: 1, `">="`: 2	longest: `GE`
`10.5`	`Integer`: 2 (`10`), `Float`: 4 (`10.5`)	longest: `FLOAT(10.5)`

Look at the last row. The integer rule was perfectly happy with `10`, but the float rule could go further, so the float rule won. The lexer had to read the `.` and the `5` before it could know. That is exactly the "one step is not always enough" situation from Section 2, handled automatically.

Key insight. Longest match is why `x==y` works with no special code: at the first `=`, rule `"=="` matches 2 characters and rule `"="` matches 1, so `==` wins.

4.4 The sign question: why `-3.5` is two tokens

This is the single most common point of confusion, so let us settle it. With the spec above, what does the lexer produce for `return -3.5 + count_2;`?

```
RETURN  MINUS  FLOAT(3.5)  PLUS  ID("count_2")  SEMI
```

Six tokens. `-3.5` is *not* one token, because the `Float` rule is `{Digit}+ "." {Digit}+` and has no sign. At the `-`, the float rule does not match at all (it needs a digit first), so the only match is the operator rule for `-`. Then `3.5` lexes as a float on its own.

"But surely negative three point five is one number?" To a human, yes. To the lexer, tokens are exactly what the spec's regexes say, never what looks

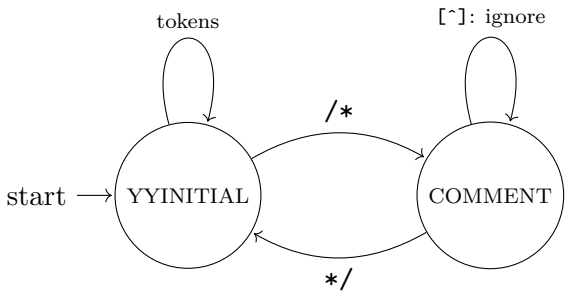
natural. And there is a good reason the spec is written this way. Consider `x-1`. If the lexer absorbed signs into number literals, longest match would produce `ID("x") INT(-1)`, and the subtraction would silently disappear. The parser cannot recover from that. So numeric literals are almost always unsigned in the lexer, and negation is an operator that the parser handles, where it has the context to know whether `-` is binary subtraction or unary negation. (The `Integer` rule in the slide spec does allow a leading `-`, which makes `x-1` lex wrongly. Treat that as a deliberate trap to notice, not a pattern to copy.)

4.5 Lexer states: different rules in different contexts

Some tokens cannot be recognised with one fixed set of rules. Inside a comment, the characters `if` are not the `IF` keyword; they are two characters to ignore. Inside a string, a space is data, not a separator. The rule set that applies depends on *where we are*.

A **lexer state** is a named, separate rule set. At any moment exactly one state is active. The action `yybegin(S)` switches the active state to `S`. `YYINITIAL` is the default state the lexer starts in.

```
<YYINITIAL> {
    "/""*"    { yybegin(COMMENT); }     // enter the comment
    "if"      { return new Token(TokenType.IF); }
    ...
}
<COMMENT> {
    "*/"      { yybegin(YYINITIAL); }   // exit the comment
    [^]       { /* ignore: any character, even newlines */ }
}
```



Read the COMMENT state as: "while in a comment, if you see `*/`, leave; otherwise eat one character and stay." The `[^]` rule is what makes the comment able to span lines: it matches any character, newline included, one at a time. Note that while in COMMENT the `"if"` rule does not exist, so a keyword inside a comment is simply never seen.

Worked example: Tracing `x/*hi*/.`

Input	State	What happens
`x`	YYINITIAL	matches `{Identifier}`: emit `ID("x")`
`/*`	YYINITIAL → COMMENT	matches `"/""*"`: switch state, no token
`h`, `i`	COMMENT	each matches `[^]`: ignored one at a time
`*/`	COMMENT → YYINITIAL	matches `"*/"`: switch back, no token
`.`	YYINITIAL	matches `"."`: emit `DOT`

Two things deserve a second look. First, why did `x` stop before the `/`? Because `/` is not an identifier character, so the identifier rule could not extend further. Second, at the `*` of `*/`, both `"*/"` (length 2) and `[^]` (length 1) match. Longest match picks `"*/"`. If the comment had contained a lone `*` not followed by `/`, only `[^]` would match it, and the comment would continue, which is what we want.

Warning / Gotcha: Nested comments break the two-state trick

Would this handle `/* outer /* inner */ still outer */`? Let us trace. YYINITIAL sees `/*`, enters COMMENT. In COMMENT, `outer`, the space, and then `/*` are all eaten by `[^]` one character at a time (there is no `/*` rule in COMMENT, so it is just characters). Then `inner`, then `*/`: the exit rule fires and we return to YYINITIAL. But we are logically still inside the outer comment. The text `still outer */` is now lexed as code: an identifier `still`, an identifier `outer`, and then `*/`, which is probably a lexical error.

Fix: keep a nesting *depth counter* in the lexer object. Increment it on every `/*` seen while already in COMMENT, decrement on every `*/`, and return to YYINITIAL only when the depth reaches 0.

```
<YYINITIAL> "/*"  { depth = 1; yybegin(COMMENT); }
<COMMENT>   "/*"  { depth++; }
<COMMENT>   "*/"  { if (--depth == 0) yybegin(YYINITIAL); }
<COMMENT>   [^]   { }
```

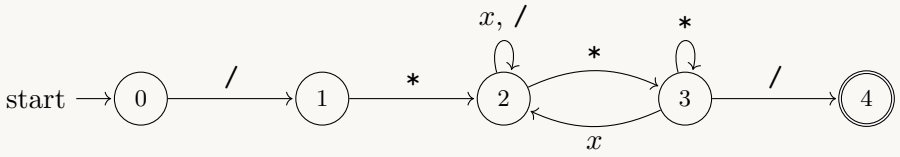
Notice what we had to do: add a piece of *memory* that a regular expression cannot express. That is no accident. "Properly nested" is the same shape as balanced parentheses, and balanced parentheses are the textbook example of a non-regular language. A finite automaton has finitely many states, so it cannot remember an unbounded nesting depth. The counter in the action is a small, controlled escape hatch one level up the Chomsky hierarchy. The regexes still do the pattern matching; the counter does the one thing they cannot.

Worked example: A single regular expression, and its DFA, for `/* ... */` comments

Lexer states are the practical way to skip comments, but a non-nested comment is a *regular* language, and exams ask for the regex and the DFA directly. Let x stand for any character other than `*` and `/`. The difficulty is the body: it may contain `*` and `/` freely, as long as `*/` never appears inside it. Group the body into chunks that can never complete a `*/`: a lone x, a lone `/`, or a run of one or more `*` followed by an x (a `*` followed by `/` would close the comment, and a `*` followed by another `*` is just a longer run). The closing is one or more `*` followed by `/`, since the body may end with extra stars, as in `/* a **/`.

$$\texttt{/}\ \texttt{*}\ \left(x \mid \texttt{/} \mid \texttt{*}^{+}x\right)^{*}\ \texttt{*}^{+}\ \texttt{/}$$

The DFA has five live states, each with a meaning:



State 0: nothing read. State 1: saw `/`. State 2: inside the body, and the last character was *not* a star, so a `/` now is harmless. State 3: inside the body, and the last character *was* a star, so a `/` now closes the comment and another `*` keeps us here. State 4: complete comment, accepting. Every missing edge (for instance x from state 0 or 1) goes to the dead state. The DFA is minimal: states 2 and 3 are distinguished by the suffix `/`, states 0 and 1 by the suffix `**/`, and 4 is the only accepting state. In a real lexer the division operator `/` also starts from state 0, and longest match plus rule priority decide between `/` and a comment, exactly as for every other overlap.

Read the regex and the DFA together: the loop on state 2 is the $(x \mid \texttt{/})$ part, the trip $2 \to 3 \to 2$ is $\texttt{*}^{+}x$, and the exit $2 \to 3 \to 4$ is $\texttt{*}^{+}\texttt{/}$. Nested comments cannot be added to this picture: a DFA cannot count how many `/*` it has passed: that is why the warning box above needed a counter.

4.6 Analysis and trade-offs

Trade-offs: Generated versus hand-written lexers

- **Performance.** A generated lexer is a table-driven finite automaton: constant work per character, linear time overall, and in practice a few nanoseconds per character. Hand-written lexers can match this but rarely do on the first attempt.
- **Correctness.** The spec is short enough to read in full. Lookahead, longest match, backtracking and buffer boundaries are handled once, in the generator, for every rule.

- **Where hand-written wins.** Languages with context that states cannot capture cleanly: Python's indentation-based `INDENT`/`DEDENT` tokens, C's "is this name a type?" ambiguity, string interpolation with nested expressions. Production compilers such as GCC, Clang and rustc use hand-written lexers for these reasons and for finer error messages. But their authors know the generated approach intimately and reproduce its discipline (peek before read, longest match) by hand.
- **When not to bother.** A configuration file with five token types is fine by hand, with `peek()`.

Summary

A lexer generator turns regex-plus-action rules into a finite-automaton tokeniser. On each call it finds the longest prefix any rule matches; if several rules match that same length, the first one written wins. Keywords therefore go before the identifier rule. Number literals are unsigned so that `x-1` keeps its minus. Lexer states give different rule sets in different contexts, such as inside comments; anything that must *count*, such as nesting, needs a counter in the actions because it is not regular.

5 Tricks, Techniques and Classic Problems

Debugging: Debugging a lexer

- **Print the token stream first.** Before debugging the parser, dump `(class, attribute, line:col)` for every token. Half of all "parser bugs" are lexer bugs, and this makes them obvious in seconds.
- **Test without whitespace.** `sum+1`, `x==y`, `a<=b`, `1..2`, `x-1`. Spacing hides lookahead bugs.
- **Test the boundaries.** Empty input; a file ending in the middle of a token; an unterminated string or comment; the longest token you can construct; a keyword that is a prefix of an identifier (`iffy`, `format`).
- **Check a regex in isolation.** `grep -E`, `python3 -c "import re; print(re.fullmatch(r'...', s))"` or https://regex101.com will show what your regex actually matches before you blame the generator.
- **JFlex**: `jflex -v spec.flex` prints the DFA size; `-dump` prints the states. **flex**: `flex -d` generates a lexer that prints each rule as it fires.

Technique: Writing specs that stay correct

- Keywords *before* the identifier rule; exact-length ties go to the first rule.
- Put a catch-all error rule *last*: `[^] { error("bad char " + yytext()); }`. Without it, the generator's default may silently echo or skip unknown input.
- Leave sign handling to the parser: `-3.5` is two tokens, so that `x-1` keeps its subtraction.

- Decode attributes in the action (parse the integer, unescape the string), so later phases never see raw text.
- Capture line and column *before* consuming the token, and update them on every character read, including inside skipped whitespace and comments.
- Anything that nests needs a counter or belongs in the parser. Do not try to write a regex for it.

Warning / Gotcha: Common student mistakes

- **Believing tokens are what look intuitive.** Tokens are exactly what the spec's regexes say. With `Float = digit+ "." digit+` and no sign, `-3.5` is `MINUS FLOAT(3.5)`, always.
- **Thinking `*` means "one or more".** It means zero or more; `a*` matches the empty string. Use `+` for one or more.
- **Thinking a character class matches a run.** `[0-9]` matches one digit. `[0-9]+` matches a run.
- **Writing recursive abbreviations** to express nesting. Nesting is not regular; use a state with a counter, or leave it to the parser.
- **Assuming the first matching rule wins.** Longest match is checked first; rule order breaks ties only.
- **Forgetting that `.` excludes newline**, so a block-comment body written with `.*` stops at the end of the first line. Use `[^]`.
- **Computing positions after the fact** by searching the source for the lexeme. Duplicates make it wrong.

Worked example: Classic exam and interview problems

P1. Classify every token in `return -3.5 + count_2;`
`return` keyword; `-` operator; `3.5` float; `+` operator; `count_2` identifier; `;` punctuation. Six tokens. `-3.5` is two tokens because the `Float` rule has no sign.

P2. Track the position. For the three lines `x = 1 / y = 2 / z = x +`, where are `y` and the final `+`?
`y`: line 2, column 1. `+`: line 3, column 7 (every character on the line counts, spaces included).

P3. What language is $(a \mid b)^*abb$? Give two members and one non-member.
Strings over $\{a, b\}$ ending in "abb". Members: "abb", "babb". Non-member: "abbb".

P4. Write a regex for C-style identifiers, which may contain digits but not start with one.
`[a-zA-Z_][a-zA-Z0-9_]*`. The first class has no digits; the second does; the star allows a one-character identifier.

P5. Why can a regex not describe balanced parentheses?
A finite automaton has finitely many states, so it cannot remember an unbounded number of unmatched open parentheses. Formally $\{(^n)^n\}$ is

not regular (pumping lemma). Balanced structure is the parser's job.

P6. A lexer has rules `"<"`, `"<="`, `"«"`, `"«="`. How does it tokenise `a«=b` and `a< =b`?

`a«=b`: at `<`, longest match takes `«=` (3 characters) over `«` (2) and `<` (1); tokens `ID LSHIFTEQ ID`. `a< =b`: only `<` matches, because the space stops every longer rule; then `=`. Tokens `ID LT ASSIGN ID`.

P7. Give a hand-lexer bug that only appears without whitespace, and its fix.

Reading with `read()` and breaking when the character is not an identifier character consumes the following operator: `sum+1` loses the `+`. Fix: `peek()` before `read()`.

P8. With `Integer = "0" | "-"?[1-9]{Digit}*`, what happens to `007`?

At the first `0`, only the `"0"` alternative matches, for 1 character; `[1-9]` rejects a second `0`. So `INT(0)`. The remaining input `07` gives `INT(0)` again, then `INT(7)`. Three tokens; the parser then rejects adjacent integers. Leading zeros were excluded on purpose (in C, `007` is octal).

P9. Does the spec need a rule for `else` if it already has one for `if`?

Yes. Each keyword is its own rule. Without one, `else` lexes as `ID("else")` and the parser will treat it as a variable name. The lexer knows only what the spec says.

Summary: Chapter 2 in seven lines

1. Lexing turns characters into tokens: class, decoded attribute, and start position. Whitespace goes; comments usually go.
2. Hand-written lexers fail by consuming before checking. Peek before read; sometimes backtrack to the last complete match.
3. A regular expression names a set of strings. Five constructs; everything else is sugar. Star includes the empty string.
4. Abbreviations are macros; recursion is banned because it would make the lexer context-free.
5. Generators resolve conflicts by longest match first, then rule order. Keywords go before identifiers.
6. Number literals are unsigned in the lexer so that `x-1` keeps its minus; negation belongs to the parser.
7. Lexer states handle context such as comments; counting (nesting) needs a counter because it is not regular.